

Prompt Engineering

By

Anand Kulkarni

anand.kulkarni1@zensar.com

Table of Content

Module	Topic
Module 1:	AI prompt basics
Module 2:	Types of AI prompt
Module 3:	Need of Prompt engineering
Module 4:	Creating effective prompts
Module 5:	Steps in Prompt Engineering
Module 6:	How Model Learns from your Prompts?

What is an AI prompt?

- Prompt is an input or a query provided to a large language model (LLM)—to generate specific responses.
- Prompt can take various forms, such as question, command, or statement, and are designed to guide the AI in producing relevant and accurate outputs.
- The effectiveness of the response depends heavily on how well the prompt is crafted.

Types of AI prompt

- **Text Generation:** Creating articles, stories, or poems.
- **Code Generation:** Writing or debugging programming code.
- **Summarization:** Condensing lengthy texts into concise summaries.
- **Translation:** Converting text between languages.
- **Image Generation:** Producing visual content based on textual descriptions.
- **Dialogue Creation:** Simulating human-like conversations.

Need of Prompt engineering

Reliability:

Crafting precise prompts ensures consistent and predictable behaviour from Gen AI & Agentic AI applications, reducing errors and unexpected outputs.

Accuracy:

Well-designed prompts lead to more accurate responses and task completion, critical for high scale automations.

Costs:

Optimized prompts reduce the token usage and computational load, significantly lowering the operational expenses for large scale AI deployments.

Creating effective prompts

Prompt engineering is the process of writing, refining, and optimizing inputs to guide generative AI systems in producing specific, high-quality outputs. To create effective prompts, you need to follow below guidelines:

Single: Always focus your prompt on a single, well-defined task or question. This clarity is crucial for accurate and useful responses from Copilot.

Specific: Ensure that your instructions are explicit and detailed. Specificity leads to more applicable and precise code suggestions.

Short: While being specific, keep prompts concise and to the point. This balance ensures clarity without overloading Copilot or complicating the interaction.

Surround: Utilize descriptive filenames and keep related files open. This provides Copilot with rich context, leading to more tailored code suggestions.

Steps in Prompt Engineering

Prompt Design: Create clear and precise prompts that effectively communicate the task to the AI.

Prompt Optimization: Continuously refine prompts through testing to improve accuracy and reduce errors.

Evaluation and Testing: Analyze outputs to check performance and make necessary improvements.

Domain Adaptation: Design prompts specific to a domain for better results in specialized tasks.

Collaboration: Work with experts and teams to align prompts with project goals.

Ethical Considerations: Ensure fairness, reduce bias and maintain privacy in prompt design.

Documentation: Record prompt versions and results to track improvements and ensure consistency.

How Model Learns from your Prompts?

AI models trained on vast amounts of data. To enhance its understanding of specific code contexts, engineers often provide it with examples. This practice, commonly found in machine learning, led to different training approaches such as:

- Zero-shot learning
- One-shot learning
- Few-shot learning

How Model Learns from your Prompts?

- Zero-shot learning

Here, model generates output without any specific examples, relying solely on its foundational training. For example,

I need a function to convert Celsius to Fahrenheit

def celsius_to_fahrenheit(celsius):

*fahrenheit = (celsius * 9/5) + 32*

return fahrenheit

How Model Learns from your Prompts?

- One-shot learning

With this approach, a single example is given, aiding the model in generating a more context-aware response.

Example: Function to convert Fahrenheit to Celsius

```
def fahrenheit_to_celsius(fahrenheit):
```

```
    celsius = (fahrenheit-32) * 5/9
```

```
    return celsius
```

Now I need a function to convert Celsius to Fahrenheit

How Model Learns from your Prompts?

```
def celsius_to_fahrenheit(celsius):
```

```
    fahrenheit = celsius * 9/5 + 32
```

```
    return fahrenheit
```

How Model Learns from your Prompts?

- Few-shot learning

In this method, Model is presented with several examples, which strike a balance between zero-shot unpredictability and the precision of fine-tuning.

Example 1: Greeting message for morning

Input: 9 AM

Output: "Good morning!"

Example 2: Greeting message for afternoon

Input: 2 PM

Output: "Good afternoon!"

How Model Learns from your Prompts?

- Few-shot learning

Example 3: Greeting message for evening

Input : 7 PM

Output : "Good evening!"

Now, generate a python code that takes the current time as input using the datetime module and returns the appropriate greeting message

Thank you!!